

Implementation of Object-Oriented Programming Concepts Using a Library Management System in Java

Experiment No: 02

Date: ____ / ____ / 2026

Title: Implementation of Object-Oriented Programming Concepts Using a Library Management System in Java

■ Objective / Aim

To design and implement a Library Management System in Java demonstrating the fundamental Object-Oriented Programming concepts such as classes and objects, constructors, encapsulation, inheritance, polymorphism, abstraction, and interfaces.

■ Mapped Outcomes

Course Outcome (CO): CO2

Program Outcomes (POs): PO1, PO5, PO9, PO10

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Create a **Book** class with Book ID, Book Name, Author, and Price.
2. Create default and parameterized constructors for initializing Book objects.
3. Implement encapsulation by declaring data members as private and accessing them through getter and setter methods.
4. Create a **Person** class and derive **Student** and **Faculty** classes using inheritance.
5. Demonstrate method overloading using different area() methods.
6. Demonstrate method overriding using **Vehicle**, **Car**, and **Bike** classes.
7. Create an abstract **Shape** class containing the abstract method draw() and implement it in **Circle** and **Rectangle**.

8. Create a **Printable** interface and implement it in the **Report** class.
9. Create objects and call the methods from the `main()` method.
10. Execute the program and verify the output.

■ Source Code

```
package task2;

// ----- Part A & B: Classes, Objects, Constructors, Encapsulation -----

class Book {

    private int bookId;
    private String bookName;
    private String author;
    private double price;

    // Default Constructor
    Book() {
        bookId = 0;
        bookName = "Unknown";
        author = "Unknown";
        price = 0.0;
    }

    // Parameterized Constructor
    Book(int bookId, String bookName, String author, double price) {
        this.bookId = bookId;
        this.bookName = bookName;
        this.author = author;
        this.price = price;
    }

    // Getter Methods
    public int getBookId() {
        return bookId;
    }

    public String getBookName() {
        return bookName;
    }

    public String getAuthor() {
        return author;
    }

    public double getPrice() {
        return price;
    }

    // Setter Methods
    public void setBookId(int bookId) {
        this.bookId = bookId;
    }

    public void setBookName(String bookName) {
        this.bookName = bookName;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    // Display Book Details
    public void displayBook() {
        System.out.println("Book ID : " + bookId);
        System.out.println("Book Name : " + bookName);
        System.out.println("Author : " + author);
        System.out.println("Price : " + price);
    }
}

// ----- Part C: Inheritance -----

class Person {

    protected String name;
```

```

        protected int age;

        Person(String name, int age) {
            this.name = name;
            this.age = age;
        }

        public void displayPerson() {
            System.out.println("Name : " + name);
            System.out.println("Age : " + age);
        }
    }

    // Student inherits Person
    class Student extends Person {

        private int rollNumber;

        Student(String name, int age, int rollNumber) {
            super(name, age);
            this.rollNumber = rollNumber;
        }

        public void displayStudent() {
            displayPerson();
            System.out.println("Roll Number : " + rollNumber);
        }
    }

    // Faculty inherits Person
    class Faculty extends Person {

        private String subject;

        Faculty(String name, int age, String subject) {
            super(name, age);
            this.subject = subject;
        }

        public void displayFaculty() {
            displayPerson();
            System.out.println("Subject : " + subject);
        }
    }

    // ----- Part D: Method Overloading -----

    class AreaCalculator {

        // Area of Circle
        public double area(double radius) {
            return Math.PI * radius * radius;
        }

        // Area of Rectangle
        public double area(double length, double breadth) {
            return length * breadth;
        }

        // Area of Square
        public double area(int side) {
            return side * side;
        }
    }

    // ----- Part D: Method Overriding -----

    class Vehicle {

        public void display() {
            System.out.println("This is a Vehicle");
        }
    }

    // Car overrides display()
    class Car extends Vehicle {

        @Override
        public void display() {
            System.out.println("This is a Car");
        }
    }

    // Bike overrides display()
    class Bike extends Vehicle {

        @Override
        public void display() {

```

```

        System.out.println("This is a Bike");
    }
}

// ----- Part E: Abstraction -----

abstract class Shape {

    abstract void draw();
}

// Circle implements draw()
class Circle extends Shape {

    @Override
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

// Rectangle implements draw()
class Rectangle extends Shape {

    @Override
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

// ----- Part E: Interface -----

interface Printable {

    void print();
}

// Report implements Printable
class Report implements Printable {

    @Override
    public void print() {
        System.out.println("Printing Library Report...");
    }
}

// ----- Main Class -----

public class LibraryManagementSystem {

    public static void main(String[] args) {

        System.out.println("=====");
        System.out.println("        LIBRARY MANAGEMENT SYSTEM");
        System.out.println("=====");

        // ----- Part A: Classes and Objects -----

        System.out.println("\n--- Part A: Classes and Objects ---");

        Book book1 = new Book();

        Book book2 = new Book(
            101,
            "Java Programming",
            "James Gosling",
            550.00
        );

        System.out.println("\nBook 1 Details:");
        book1.displayBook();

        System.out.println("\nBook 2 Details:");
        book2.displayBook();

        // ----- Part B: Encapsulation -----

        System.out.println("\n--- Part B: Encapsulation ---");

        book1.setBookId(102);
        book1.setBookName("Object Oriented Programming");
        book1.setAuthor("Herbert Schildt");
        book1.setPrice(650.00);

        System.out.println("\nBook 1 Details After Using Setters:");

        System.out.println("Book ID   : " + book1.getBookId());
        System.out.println("Book Name : " + book1.getBookName());
        System.out.println("Author   : " + book1.getAuthor());
    }
}

```

```

        System.out.println("Price      : " + book1.getPrice());

// ----- Part C: Inheritance -----

System.out.println("\n--- Part C: Inheritance ---");

Student student = new Student(
    "Rahul",
    19,
    101
);

System.out.println("\nStudent Details:");
student.displayStudent();

Faculty faculty = new Faculty(
    "Dr. Kumar",
    40,
    "Java Programming"
);

System.out.println("\nFaculty Details:");
faculty.displayFaculty();

// ----- Part D: Method Overloading -----

System.out.println("\n--- Part D: Method Overloading ---");

AreaCalculator calculator = new AreaCalculator();

System.out.println("Area of Circle = "
    + calculator.area(5.0));

System.out.println("Area of Rectangle = "
    + calculator.area(10.0, 5.0));

System.out.println("Area of Square = "
    + calculator.area(5));

// ----- Part D: Method Overriding -----

System.out.println("\n--- Part D: Method Overriding ---");

Vehicle vehicle;

vehicle = new Car();
vehicle.display();

vehicle = new Bike();
vehicle.display();

// ----- Part E: Abstraction -----

System.out.println("\n--- Part E: Abstraction ---");

Shape circle = new Circle();
Shape rectangle = new Rectangle();

circle.draw();
rectangle.draw();

// ----- Part E: Interface -----

System.out.println("\n--- Part E: Interface ---");

Printable report = new Report();

report.print();

System.out.println("\n=====");
System.out.println("        PROGRAM COMPLETED");
System.out.println("=====");
    }
}

```

■ Compilation & Execution Commands

Save the source code exactly as (inside a folder named task2):

```
task2/LibraryManagementSystem.java
```

Compile the program:

```
javac task2/LibraryManagementSystem.java
```

Run the compiled program:

```
java task2.LibraryManagementSystem
```

■ Test Cases & Execution Data

Test Case	Input / Operation	Expected Result	Status
TC-01	Create "Book" objects	Book details displayed	Passed
TC-02	Use getter/setter methods	Updated book details displayed	Passed
TC-03	Create Student and Faculty	Inherited properties displayed	Passed
TC-04	Call overloaded "area()" methods	Areas of shapes displayed	Passed
TC-05	Create Car and Bike using Vehicle reference	Overridden methods executed	Passed
TC-06	Create Circle and Rectangle objects	"draw()" methods executed	Passed
TC-07	Create Report object	Report message displayed	Passed

■ Output Screen

Expected Output

```
=====
      LIBRARY MANAGEMENT SYSTEM
=====

--- Part A: Classes and Objects ---

Book 1 Details:
Book ID   : 0
Book Name : Unknown
Author    : Unknown
Price     : 0.0

Book 2 Details:
Book ID   : 101
Book Name : Java Programming
Author    : James Gosling
Price     : 550.0

--- Part B: Encapsulation ---

Book 1 Details After Using Setters:
Book ID   : 102
Book Name : Object Oriented Programming
Author    : Herbert Schildt
Price     : 650.0

--- Part C: Inheritance ---

Student Details:
```

Name : Rahul
Age : 19
Roll Number : 101

Faculty Details:
Name : Dr. Kumar
Age : 40
Subject : Java Programming

--- Part D: Method Overloading ---

Area of Circle = 78.53981633974483
Area of Rectangle = 50.0
Area of Square = 25.0

--- Part D: Method Overriding ---

This is a Car
This is a Bike

--- Part E: Abstraction ---

Drawing a Circle
Drawing a Rectangle

--- Part E: Interface ---

Printing Library Report...

```
=====
      PROGRAM COMPLETED
=====
```

Eclipse Console output screenshots captured after successful execution:

```

<terminated> LibraryManagementSystem [Java Applic
=====
LIBRARY MANAGEMENT SYSTEM
=====

--- Part A: Classes and Objects ---

Book 1 Details:
Book ID   : 0
Book Name : Unknown
Author    : Unknown
Price     : 0.0

Book 2 Details:
Book ID   : 101
Book Name : Java Programming
Author    : James Gosling
Price     : 550.0

--- Part B: Encapsulation ---

Book 1 Details After Using Setters:
Book ID   : 102
Book Name : Object Oriented Programming
Author    : Herbert Schildt
Price     : 650.0

--- Part C: Inheritance ---

Student Details:
Name      : Rahul
Age       : 19
Roll Number : 101

Faculty Details:
Name      : Dr. Kumar
Age       : 40
Subject   : Java Programming

```

Screenshot 1: Eclipse Console Output

```

--- Part B: Encapsulation ---

Book 1 Details After Using Setters:
Book ID   : 102
Book Name : Object Oriented Programming
Author    : Herbert Schildt
Price     : 650.0

--- Part C: Inheritance ---

Student Details:
Name      : Rahul
Age       : 19
Roll Number : 101

Faculty Details:
Name      : Dr. Kumar
Age       : 40
Subject   : Java Programming

--- Part D: Method Overloading ---
Area of Circle = 78.53981633974483
Area of Rectangle = 50.0
Area of Square = 25.0

--- Part D: Method Overriding ---
This is a Car
This is a Bike

--- Part E: Abstraction ---
Drawing a Circle
Drawing a Rectangle

--- Part E: Interface ---
Printing Library Report...

=====
PROGRAM COMPLETED
=====

```

Screenshot 2: Eclipse Console Output

```

LIBRARY MANAGEMENT SYSTEM
=====

--- Part A: Classes and Objects ---

Book 1 Details:
Book ID   : 0
Book Name : Unknown
Author    : Unknown
Price     : 0.0

Book 2 Details:
Book ID   : 101
Book Name : Java Programming
Author    : James Gosling
Price     : 550.0

--- Part B: Encapsulation ---

Book 1 Details After Using Setters:
Book ID   : 102
Book Name : Object Oriented Programming
Author    : Herbert Schildt
Price     : 650.0

--- Part C: Inheritance ---

Student Details:
Name      : Rahul
Age       : 19
Roll Number : 101

Faculty Details:
Name      : Dr. Kumar
Age       : 40
Subject   : Java Programming

--- Part D: Method Overloading ---
Area of Circle = 78.53981633974483
Area of Rectangle = 50.0
Area of Square = 25.0

```

Screenshot 3: Eclipse Console Output

```

JavaBas... Problems Javadoc Declaration Console X
<terminated> LibraryManagementSystem [Java Application] C:\Users\...

--- Part B: Encapsulation ---

Book 1 Details After Using Setters:
Book ID   : 102
Book Name : Object Oriented Programming
Author    : Herbert Schildt
Price     : 650.0

--- Part C: Inheritance ---

Student Details:
Name      : Rahul
Age       : 19
Roll Number : 101

Faculty Details:
Name      : Dr. Kumar
Age       : 40
Subject   : Java Programming

--- Part D: Method Overloading ---
Area of Circle = 78.53981633974483
Area of Rectangle = 50.0
Area of Square = 25.0

--- Part D: Method Overriding ---
This is a Car
This is a Bike

--- Part E: Abstraction ---
Drawing a Circle
Drawing a Rectangle

--- Part E: Interface ---
Printing Library Report...

=====
PROGRAM COMPLETED
=====

```

Screenshot 4: Eclipse Console Output

■ Learning Outcome / Result

Successfully implemented and verified a Library Management System using Java. The experiment demonstrated classes and objects, constructors, encapsulation, inheritance, method overloading, method overriding, polymorphism, abstraction, and interfaces. The program improved understanding of object-oriented programming, code reusability, data hiding, and modular program design.